Arab Academy for Science, Technology & Maritime Transport
College of Computing & Information Technology
Department of Cybersecurity

ForensiX

Autonomous Forensic AI Framework

ForensiX Project Report

AI in Cybersecurity

[CCY4303]

Project Team

Ahmed Aamer	**Youssef Hazem**
Ali Hesham	**Mohamed Ahmed**

Supervised by
Dr. Mohamed Ezzat Elhamahmy

Academic Year 2024 – 2025

TABLE OF CONTENTS

Contents

LIST OF FIGURES

1. Executive Summary

The rapid evolution of cyber threats and the sheer volume of forensic evidence generated by modern incidents have severely strained the capacity of digital forensics teams. Manual orchestration of tools, correlation of disparate data sources, and narrative report generation remain labor-intensive, error-prone, and difficult to scale. This project presents ForensiX, a fully implemented, end-to-end agentic AI framework that autonomously conducts forensic artifact analysis and generates professional, explainable incident reports.

ForensiX accepts forensic artifacts including memory dumps (.vmem/.raw), portable executable files (.exe/.dll), network packet captures (.pcap/.pcapng), and Windows Event Logs (.evtx) and orchestrates a dynamic, reasoning-driven investigative pipeline. The system employs a ReAct (Reasoning + Acting) agent loop in which a large language model (LLM) iteratively selects and executes appropriate forensic tools based on accumulated findings, without any analyst intervention.

Key capabilities include mandatory Shannon entropy classification, YARA rule matching, Volatility3 memory analysis, string extraction, and optional VirusTotal enrichment.

The final correlator module synthesizes all tool outputs into three ranked attack hypotheses with confidence scores, a MITRE ATT&CK-annotated forensic timeline, a severity-classified suspicious strings table, and an executive summary paragraph. An adversary attribution engine maps discovered techniques to six known threat actor profiles (APT28, APT29, Lazarus Group, FIN7, REvil, and Cobalt Strike operators) using rule-based TTP lookups with no additional LLM cost. A rule-based confidence scorer aggregates all findings into a 0–100 risk score.

The system is deployable via a single Docker Compose command and supports both cloud-based Claude Sonnet and local Ollama LLaMA 3.2 backends, switchable live without restart.

The frontend delivers a five-page React 18 application featuring real-time WebSocket streaming of agent reasoning steps, an animated risk gauge, a 14-tactic MITRE ATT&CK heatmap, an interactive SVG threat graph, and one-click PDF report download.

ForensiX represents a significant advancement from tool-assisted to agent-assisted forensics, lowering the barrier for junior analysts while enabling senior experts to focus on strategic decision-making.

2. Introduction and Background

Digital forensics is a cornerstone of modern cybersecurity incident response. When a breach occurs, forensic analysts must collect, preserve, and analyze evidence across a wide array of artifact types memory snapshots, network captures, executable binaries, and system logs. Each artifact type demands specialized tooling, and correlating findings across these sources into a coherent attack narrative requires significant expertise and time.

Contemporary forensic pipelines suffer from three fundamental limitations. First, tool orchestration is entirely manual: analysts must individually invoke tools such as Volatility3 for memory analysis, strings and binwalk for binary inspection, and YARA for signature matching, then mentally integrate their outputs. Second, context correlation linking a suspicious process identifier to a network connection and subsequently to a registry persistence mechanism is a manual, error-prone endeavor. Third, scalability is constrained by human throughput; modern memory dumps can reach terabytes in size, and the volume of forensic cases continues to grow.

Prior work in forensic automation has largely focused on narrow, tool-specific tasks (e.g., detecting a specific rootkit family) rather than holistic, end-to-end investigation. Recent advances in LLM-based agents (exemplified by frameworks like AutoGPT and ReAct) demonstrate that language models can effectively reason about tool selection and interpret intermediate results. However, these general-purpose agents have not been specialized for the rigorous, evidence-based requirements of digital forensics, where hallucinated findings can have serious legal and operational consequences.

ForensiX addresses this gap by combining a disciplined, budget-constrained agentic loop (maximum 10 LLM calls per job) with deterministic, rule-based components for adversary attribution and confidence scoring. This design philosophy ensures that the AI augments forensic reasoning without introducing unbounded hallucination risk, and that the system remains practical and cost-effective in real operational environments.

3. Problem Statement

The core problem ForensiX addresses is the disconnect between raw forensic data extraction and actionable investigative synthesis.

Despite the availability of mature forensic tools, analysts continue to face three critical challenges:

3.1 Manual Tool Orchestration

Forensic investigations require analysts to manually select, configure, and execute a heterogeneous set of tools depending on the artifact type. This process is slow, requires significant expertise, and prevents scaling to handle the volume of modern incidents. A junior analyst lacking knowledge of the correct tool sequence may miss critical evidence.

3.2 Context Correlation Complexity

Connecting indicators across tool outputs for example, linking a suspicious process identified by Volatility3 to a network beaconing connection found in a PCAP, and then to a YARA-matched malware signature

requires the analyst to maintain a comprehensive mental model of the investigation. This cross-tool correlation is where critical evidence is most frequently overlooked.

3.3 Report Generation Overhead

The production of professional forensic reports suitable for incident response briefings, legal proceedings, or executive review is an extremely time-consuming task. Analysts must manually compile findings, construct timelines, and write narrative explanations work that can consume as much time as the analysis itself and that does not leverage the analyst's core investigative skills.

Together, these challenges result in significant analyst burnout, case backlogs, inconsistent report quality, and delayed incident response. ForensiX targets all three simultaneously through an integrated agentic framework.

4. Literature Review and Gap Analysis

Research in automated digital forensics has historically focused on narrow, tool-specific automation. Plaso (Log2Timeline) automates the extraction and normalization of timestamps across multiple artifact types but produces raw timelines requiring manual interpretation. Autopsy and its underlying Sleuth Kit framework provide comprehensive disk analysis automation, yet still rely on human analysts to synthesize findings into investigative narratives.

In the domain of memory forensics, Volatility3 provides a powerful plugin architecture but requires manual plugin selection and result interpretation. YARA rule matching similarly depends on analysts maintaining and applying curated rule sets. Neither tool provides reasoning about which forensic action should follow from a given intermediate finding.

The emergence of LLM-based agent frameworks notably ReAct (Yao et al., 2023) [1], which interleaves reasoning and action steps provides a compelling foundation for agentic forensics. AutoGPT [13] and similar systems, built with composable LLM toolkits such as LangChain [8], have demonstrated that LLMs can autonomously select and invoke tools, but these systems were not designed for the evidence-chain rigor required by forensics and are prone to hallucination in high-stakes analytical contexts [7].

The concept of applying ReAct-style agent loops to digital forensics has been discussed informally in the security research community, but no peer-reviewed implementation addressing LLM cost budgeting, context window management, adversary attribution, confidence scoring, or production-quality report generation has been published [see ref. 13 for the closest related agent framework].

ForensiX fills this gap by delivering a fully implemented system that incorporates (1) a cost-bounded ReAct loop with tool output truncation to prevent context overflow, (2) deterministic rule-based components for attribution and scoring that eliminate hallucination risk in those critical functions, (3) MITRE ATT&CK integration for structured technique mapping, and (4) a production-ready frontend with real-time agent transparency through streaming reasoning display.

5. Research Objectives

The primary goal of this project is to design, implement, and evaluate ForensiX as a fully functional agentic AI framework for autonomous digital forensic investigation. The following specific objectives guide the system design and implementation:

- Autonomous tool orchestration via a ReAct agent loop with a strictly bounded LLM budget of a maximum of 10 API calls per forensic job, ensuring cost-predictable and scalable operation.
- Multi-format artifact support covering memory dumps (.vmem/.raw), PE executables (.exe/.dll), PCAP network captures (.pcap/.pcapng), Windows Event Logs (.evtx), and plain log files each routed to the appropriate forensic tool subset by a dedicated file type router.
- Mandatory Shannon entropy classification as a zero-cost pre-screening step executed before any LLM call, providing artifact-level encryption and compression state before the agent loop begins.
- MITRE ATT&CK-annotated forensic timeline and interactive 14-tactic heatmap, mapping discovered technique IDs to ATT&CK tactics and surfacing coverage gaps across the kill chain.
- Rule-based adversary attribution against six hardcoded threat actor profiles (APT28, APT29, Lazarus Group, FIN7, REvil, and Cobalt Strike operators) consuming zero LLM tokens, ensuring hallucination-free attribution results.
- Automated ReportLab PDF incident report generation with one-click download, producing a professional deliverable including a dark cover page, risk confidence bars, the forensic timeline, suspicious strings table, and complete agent reasoning trace.

6. System Architecture

ForensiX is structured as a three-layer system: a containerized forensic tooling environment, a Python/FastAPI backend pipeline, and a React 18 frontend. The following subsections describe each layer in detail.

6.1 High-Level Pipeline Overview

Upon artifact upload, the system executes the following end-to-end pipeline:

- The user uploads a forensic artifact via drag-and-drop or the 'Load Demo Sample' button on the Upload Page.
- python-magic performs file type detection and routes the artifact to the appropriate tool set.
- Shannon entropy analysis executes immediately mandatory, zero LLM cost classifying the artifact by encryption/compression state.
- The ReAct agent loop begins: the LLM receives cumulative findings and selects the next tool. Up to 9 tool-selection steps are allowed (max 9 LLM calls).
- A final Correlator LLM call synthesizes all findings into hypotheses, a timeline, suspicious strings, and an executive summary.
- Rule-based AdversaryProfiler and ConfidenceScorer modules enrich the results without additional LLM calls.
- Optional VirusTotal enrichment runs if VT_API_KEY is configured.
- The PDF report is generated by ReportLab and stored. The job is persisted in SQLite.

Figure 1: *ForensiX Upload page showing the drag-and-drop zone, AI mode toggle (Claude / Ollama), and Load Demo Sample button*

6.2 Backend Architecture

The backend is implemented in Python 3.11 using the FastAPI framework and consists of the following core modules:

Module	Responsibility
main.py	FastAPI application entry point. CORS configuration, PDF/preview endpoints, live AI mode switch (POST /api/ai-mode).
router.py	python-magic file type detection and artifact routing. Determines which tool subset is invoked based on detected MIME type (previously mis-attributed to main.py).
health.py	Startup AI backend reachability check. Verifies the configured LLM endpoint is reachable before accepting jobs.
llm_client.py	Unified LLM interface supporting Claude Sonnet and Ollama LLaMA 3.2. Provides get_mode/set_mode. Raises RuntimeError if the Anthropic API key is missing or invalid when claude mode is active — no silent fallback.
executor.py	Core agent loop. Enforces mandatory entropy-first execution. MAX_AGENT_STEPS = 9. Emits llm_reason WebSocket events for each reasoning step.
selector.py	Executes one LLM call per agent step. Returns {next_tool, reasoning} JSON to the executor.
correlator.py	Final synthesis LLM call. Produces 3 ranked attack hypotheses, a MITRE-annotated timeline, up to 10 suspicious strings with severity, and an executive summary.

Module	Responsibility
adversary.py	Rule-based TTP-to-actor matching. No LLM call. Maps discovered techniques to 6 hardcoded threat actor profiles.
confidence.py	Rule-based 0–100 risk scorer. Aggregates tool success rates, IOC counts, YARA severity, entropy classification, and Volatility findings.
vt_client.py	Optional VirusTotal enrichment. Activates when VT_API_KEY environment variable is set.
pdf_builder.py	ReportLab PDF generation. Dark cover page, logo, confidence bars, suspicious strings table with severity coloring, incident timeline, executive summary, and tool execution grid.
job_store.py	In-memory job dictionary with WebSocket event buffering for reconnect replay.
db.py	SQLite persistence layer for completed case storage and retrieval.

6.3 Forensic Tool Suite

All forensic tools are containerized within Docker and require no host installation. Each tool returns a typed ToolOutput Pydantic model ensuring consistent downstream consumption:

Tool	Implementation	Findings
entropy	Pure Python, no external deps	Shannon entropy per block; classification: <5.0 benign, 5.0–6.5 compressed, 6.5–7.2 packed/obfuscated, >7.2 encrypted
strings	GNU binutils (Debian apt)	Printable strings (max 80 items); IOC candidates for enrichment
yara	yara-python pip package	YARA rule matches with byte offsets and severity ratings. Rules auto-discovered from backend/yara_rules/
volatility3	volatility3 pip; cached fallback for cridex.vmem	Process list (pslist, max 30), network connections (netscan, max 30), command lines (cmdline, max 20), DLL list, image info
binwalk	binwalk apt package	Embedded file signatures; firmware sections and embedded archive detection
pcap	tshark (Wireshark CLI, apt)	DNS query extraction, HTTP host enumeration, TCP stream reconstruction. Activated for .pcap/.pcapng artifacts.
evtx	python-evtx pip package	Windows Event Log parsing. Maps Event IDs to MITRE techniques (T1059 — Execution, T1078 — Valid Accounts, T1110 — Brute Force, and others). Activated for .evtx artifacts.

Tool output caps (strings: 80, pslist: 30, netscan: 30, cmdline: 20) are applied before the correlation step to prevent LLM context window overflow — a critical engineering constraint ensuring reliable operation at bounded cost.

6.4 LLM Budget and Cost Constraints

A defining architectural constraint of ForensiX is its strictly bounded LLM usage. The system is limited to a maximum of 10 LLM calls per forensic job: up to 9 calls for agent tool selection (one per ReAct step), and exactly 1 call for final correlation. The adversary profiler and confidence scorer are entirely rule-based, consuming zero LLM tokens. This design makes the system cost-predictable and viable for high-volume deployment without unbounded API expenditure.

The AI backend is switchable live via POST /api/ai-mode without any service restart, supporting both cloud-based Claude Sonnet and local Ollama LLaMA 3.2. If the Anthropic API key is missing or invalid when claude mode is active, llm_client.py raises a RuntimeError and the job fails with a visible error — there is no silent fallback to Ollama. This is a deliberate design decision to prevent undetected misconfiguration in production deployments.

6.5 Frontend Architecture

The frontend is implemented in React 18 with Vite and Tailwind CSS, using local useState only no external state management library. The application follows a five-page flow:

Page	Key Features
Upload	Drag-and-drop artifact upload; live AI mode toggle (Claude/Ollama); 'Load Demo Sample' button for cridex.vmem demo
LiveAgent	WebSocket terminal with exponential-backoff reconnect; server-side event buffering for replay; llm_reason events displayed as expandable purple 'THINK' rows showing LLM chain-of-thought
Results	11-section results dashboard: risk gauge, entropy histogram, MITRE heatmap, adversary card, executive summary, interactive threat graph, evidence table, suspicious strings, tool execution grid, agent reasoning log
Report	Inline PDF preview with one-click download via GET /api/jobs/{id}/report
Case History	Persistent case browser listing all completed jobs from the SQLite store. Each entry displays artifact filename, UUID, completion status badge, and detected file type. Accessible via the CASE HISTORY button in the top navigation bar.

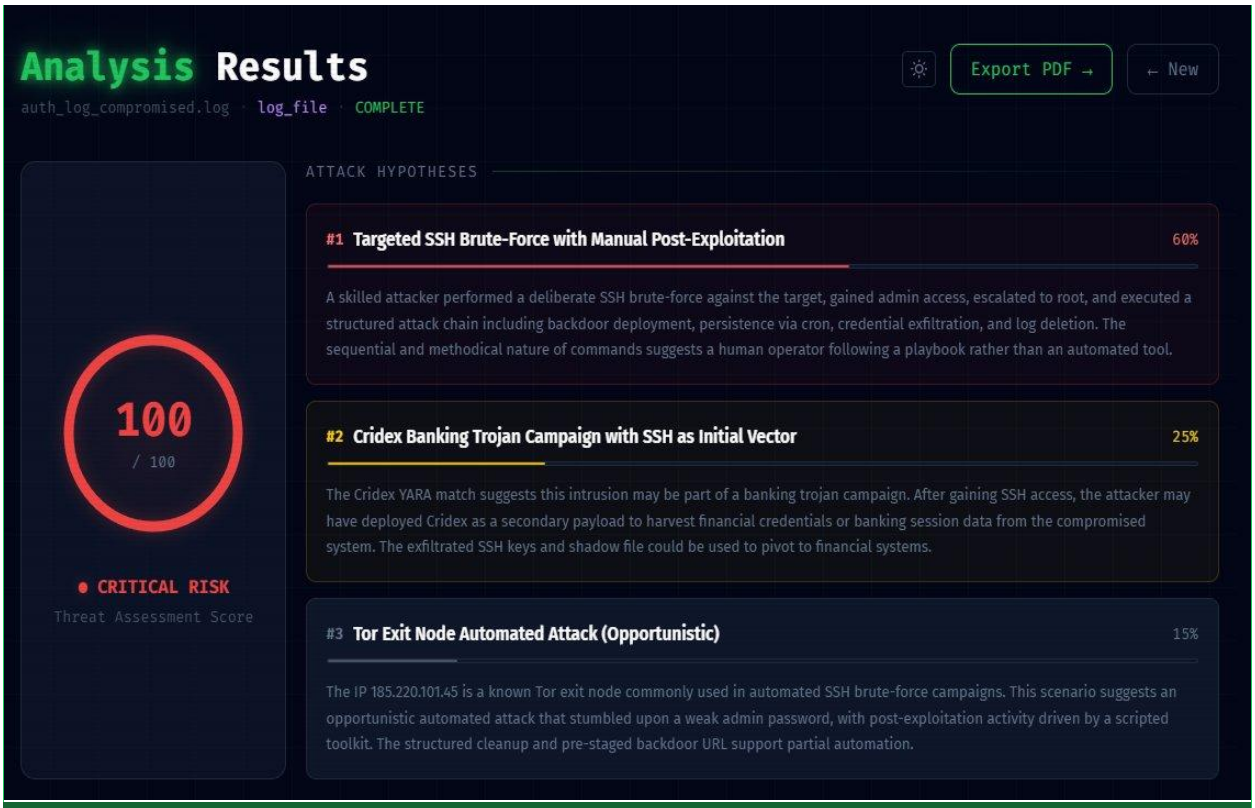

Figure 2: *ForensiX Results page showing the complete 11-section dashboard with risk gauge, MITRE heatmap, adversary card, and evidence table*

7. Proposed Methodology

7.1 ReAct Agent Loop

The investigative pipeline is driven by a ReAct (Reasoning + Acting) agent loop implemented in pipeline/executor.py. Upon artifact ingestion, the entropy analysis tool is unconditionally invoked first a deterministic, zero-cost pre-screening step. The entropy result is passed to the LLM, which responds with a JSON object specifying the next tool to invoke and its reasoning. This reasoning step is emitted as a WebSocket event (llm_reason), enabling real-time display of the agent's chain of thought in the frontend terminal.

This tool selection and execution cycle repeats until the agent invokes the special 'DONE' tool or the MAX_AGENT_STEPS limit of 9 is reached. Every agent reasoning step is logged as an AgentReasoningStep object and persisted with the job, providing a complete, auditable investigation trace.

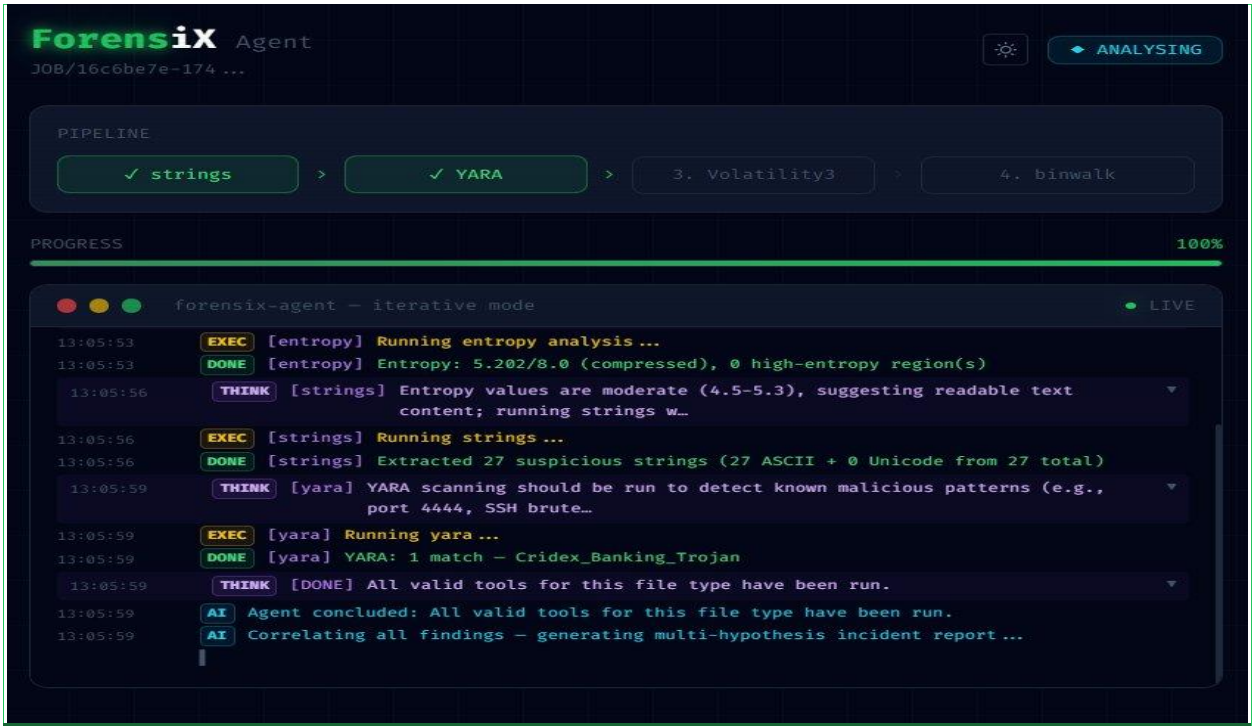

Figure 3: *WebSocket terminal mid-analysis showing tool execution logs and an expanded purple THINK row revealing the LLM's chain-of-thought reasoning*

7.2 Entropy Classification

Shannon entropy is computed per block across the artifact using pure Python (no external dependencies). The analyzer auto-scales to approximately 160 blocks for uniform coverage regardless of artifact size. Classification thresholds: entropy below 5.0 indicates a benign or plaintext artifact; 5.0–6.5 indicates compression; 6.5–7.2 indicates packing or obfuscation; and above 7.2 indicates encryption. This classification is surfaced to the frontend as a per-block histogram SVG with dashed threshold lines.

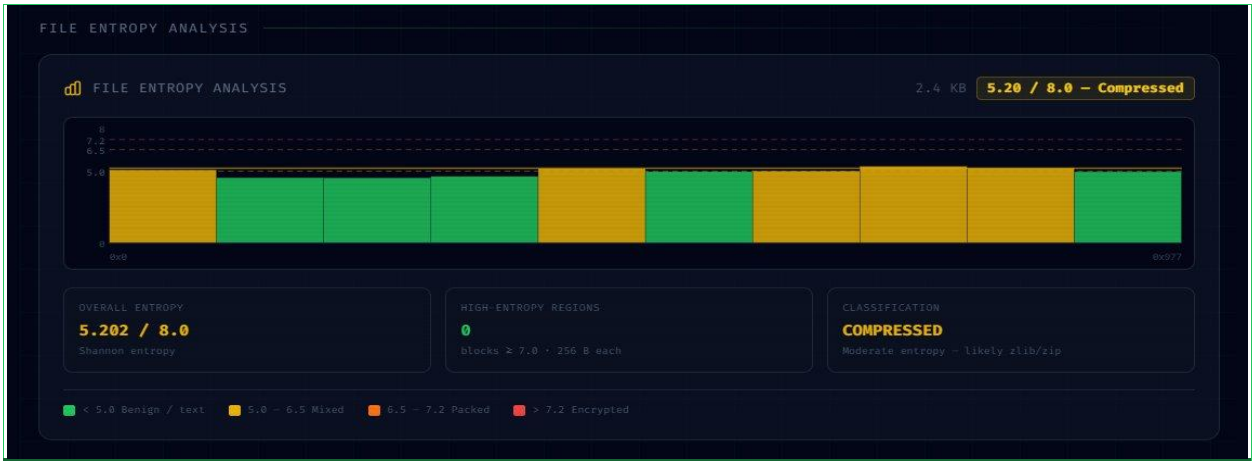

Figure 4: *EntropyChart.tsx rendering per-block Shannon entropy for the cridex.vmem demo sample, with dashed threshold lines at 5.0, 6.5, and 7.2*

7.3 MITRE ATT&CK Integration

The Correlator module includes a _extract_mitre_tactics() function that maps discovered technique IDs to MITRE ATT&CK tactics. A TECHNIQUE_TACTIC fallback dictionary (e.g., T1059 maps to Execution) ensures correct tactic assignment even when the LLM's technique naming deviates from canonical identifiers. Timeline events are annotated with both technique and tactic labels, and the frontend renders a 14-tactic heatmap where triggered tactics are highlighted in red with hover tooltips for individual techniques.

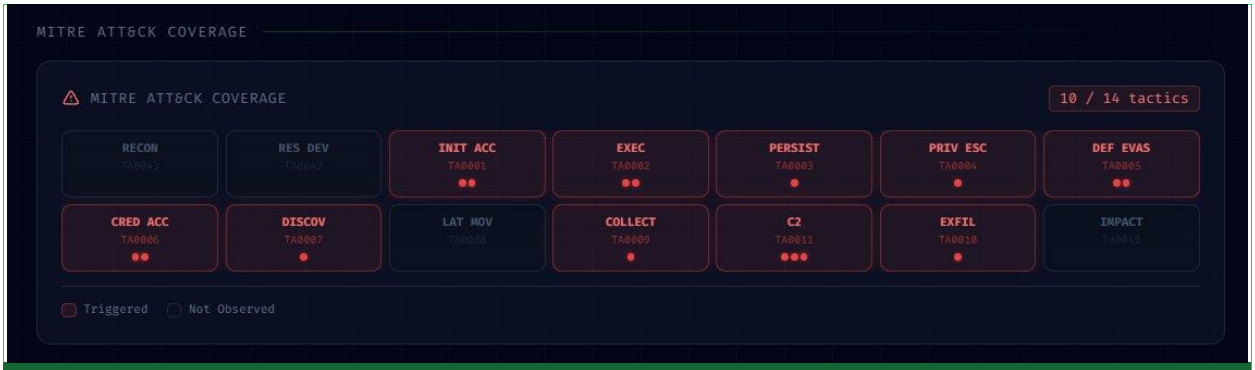

Figure 5: *MitreHeatmap.tsx showing the 14-tactic coverage grid with triggered tactics highlighted and matched technique IDs*

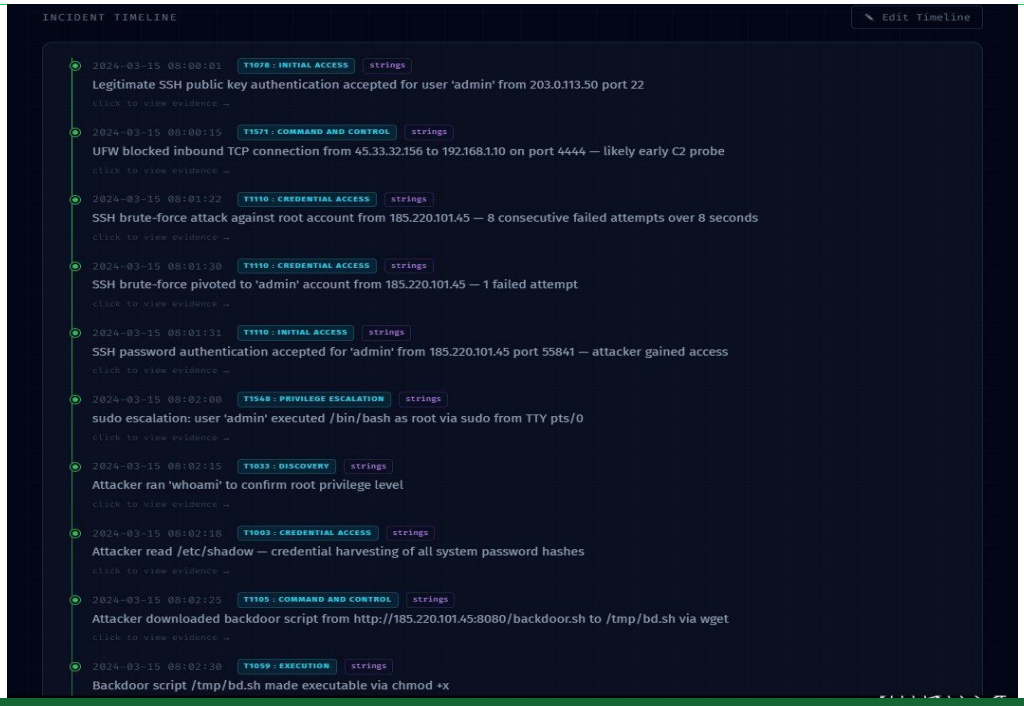

Figure 6: *Timeline.tsx rendering the MITRE ATT&CK-annotated forensic timeline. Each event shows timestamp, technique ID, tactic label, source tool badge, and a click-to-expand evidence link.*

7.4 Adversary Attribution

The AdversaryProfiler module performs rule-based TTP-to-actor matching against six hardcoded threat actor profiles: APT28, APT29, Lazarus Group, FIN7, REvil, and Cobalt Strike operators. Matching is performed by comparing the set of discovered MITRE technique IDs against each actor's known TTP fingerprint. Attribution results include the actor name, estimated motivation, confidence percentage, and matching TTP badges. This module consumes zero LLM tokens, making attribution both fast and hallucination-free.

Figure 7: *AdversaryCard.tsx displaying the matched threat actor profile with name, motivation, confidence percentage, and TTP badge grid*

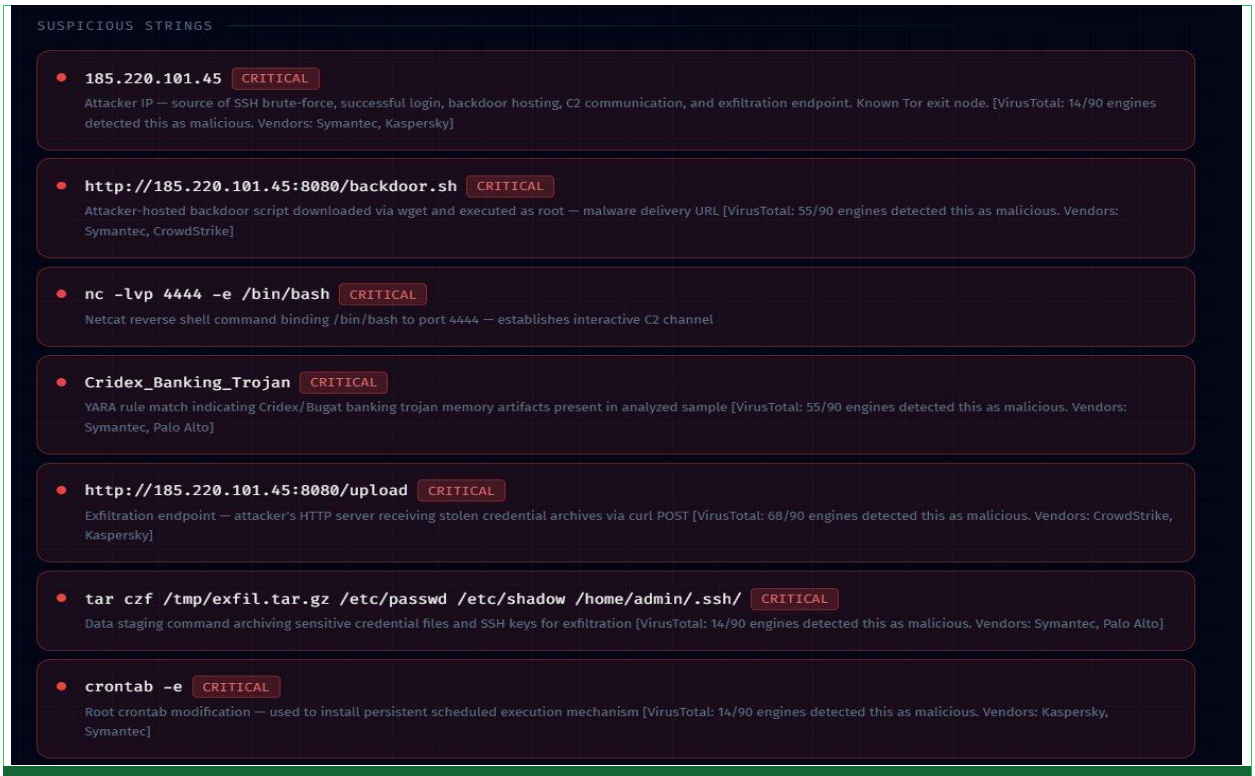

Figure 8: *Correlator output rendered as a severity-classified suspicious strings table. Each entry shows the IOC string, CRITICAL/HIGH/MEDIUM badge, analyst description, and VirusTotal engine hit count.*

7.5 Confidence Scoring

The ConfidenceScorer module computes a deterministic 0–100 risk score by aggregating multiple evidence signals: tool execution success rates, IOC count thresholds, YARA match severity levels, entropy classification, and significant Volatility findings such as suspicious network connections or injected processes. The score is displayed on the Results page as an animated SVG circular gauge and is embedded in the PDF report with a color-coded confidence bar.

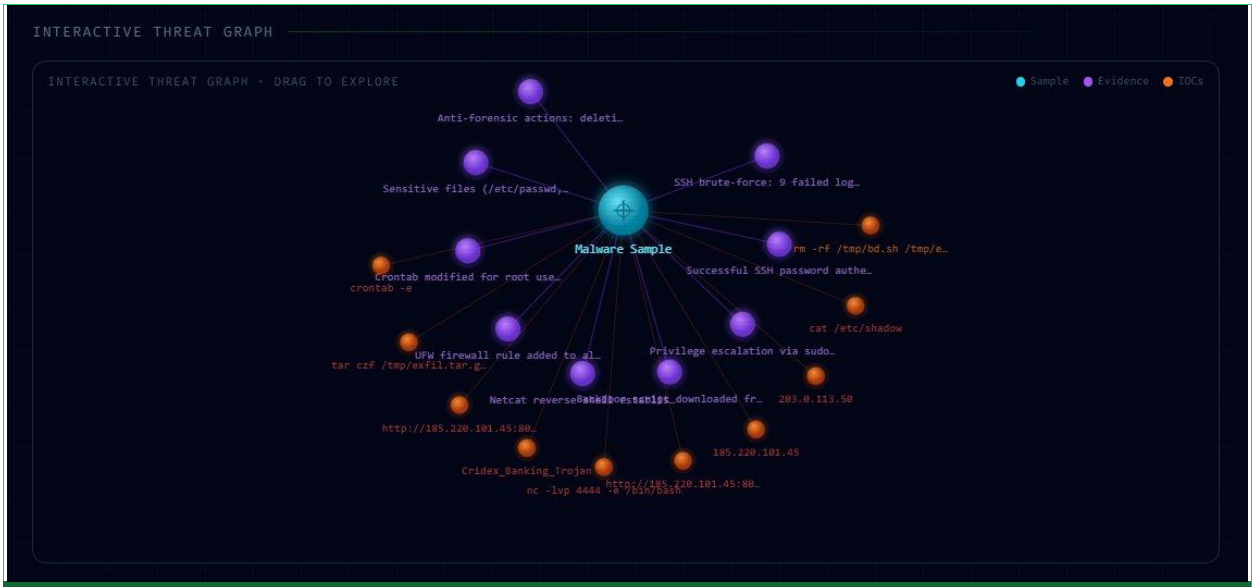

Figure 9: *Results page showing the animated SVG circular risk gauge alongside the interactive force-simulation threat graph with draggable IOC nodes*

7.6 YARA Rule Management

YARA rules are stored as .yar files in the backend/yara_rules/ directory. The system performs auto-discovery and compilation of all rules at startup, caching the compiled rule set for efficiency. New rules can be added by dropping .yar files into this directory — they are automatically picked up on the next scan without any system modification. Rule matches include byte offsets and severity metadata consumed by the correlator and confidence scorer.

7.7 Demo Sample

The system ships with a bundled demo artifact: sample/cridex.vmem, a synthetic Windows XP memory image containing realistic forensic indicators representative of the Cridex banking trojan. This includes process artifacts, C2 network indicators (TCP/8080 beaconing), and registry Run key persistence mechanisms. The 'Load Demo Sample' button on the Upload page invokes POST /api/upload-sample, which loads this artifact and launches the full pipeline. Pre-computed Volatility3 results are cached in

tools/volatility_cache.py, ensuring the demo functions correctly even in environments where Volatility3 cannot execute.

8. API Reference

The ForensiX backend exposes the following REST and WebSocket endpoints:

Method	Path	Description
POST	/api/upload	Upload forensic artifact and start analysis pipeline
POST	/api/upload-sample	Load bundled cridex.vmem demo sample and start pipeline
GET	/api/jobs/{id}	Poll job status and retrieve full results JSON
GET	/api/jobs/{id}/report	Download generated PDF report as attachment
GET	/api/jobs/{id}/report/preview	Render PDF report inline in browser
GET	/api/ai-mode	Retrieve current AI backend (claude or ollama)
POST	/api/ai-mode	Switch AI backend live without restart. Body: {"mode": "claude" | "ollama"}
WS	/ws/{job_id}	WebSocket stream for real-time agent events including llm_reason, tool_result, and job_complete

9. Deployment and Configuration

9.1 Docker Compose Deployment (Recommended)

The recommended production deployment uses Docker Compose, which containerizes both the FastAPI backend and the React frontend with all forensic tool dependencies pre-installed. After the build completes, the frontend is accessible at http://localhost:3000 and the backend API at http://localhost:8000.

```
docker compose up --build
```

9.2 Environment Variables

Variable	Default	Description
AI_MODE	claude	LLM backend selection: 'claude' or 'ollama'
ANTHROPIC_API_KEY	(none)	Required when AI_MODE=claude. Missing or invalid key raises RuntimeError — job fails visibly, no fallback.

Variable	Default	Description
OLLAMA_BASE_URL	http://host.docker.internal:11434	Base URL for the local Ollama server
OLLAMA_MODEL	llama3.2	Ollama model identifier to use for agent reasoning
VT_API_KEY	(none)	Optional. Enables VirusTotal enrichment for discovered IOCs when set.

9.3 Development Mode

- Backend only: cd backend && pip install -r requirements.txt && AI_MODE=ollama uvicorn main:app --reload
- Frontend only: cd frontend && npm install && npm run dev (proxies /api and /ws requests to port 8000)

10. Key Frontend Components

The ForensiX frontend consists of several custom-built React components, none of which rely on third-party visualization libraries all charts and graphs are implemented in raw SVG:

Component	Description
ThreatGraph.tsx	Custom SVG force-simulation threat graph. Nodes represent the artifact root, evidence items, and IOCs. Fully draggable using getScreenCTM().inverse() for Retina display compatibility. Simulation cools after 260 frames.
MitreHeatmap.tsx	14-tactic MITRE ATT&CK coverage grid. Handles technique names, IDs, short codes, aliases, and partial strings via a TECHNIQUE_TACTIC fallback map. Triggered tactics highlighted in red with hover tooltips.
EntropyChart.tsx	SVG bar chart with preserveAspectRatio='none'. Renders per-block entropy with dashed threshold lines at 5.0, 6.5, and 7.2. Hex-offset X-axis for artifact navigation.
Timeline.tsx	Vertical forensic timeline with MITRE technique badges and purple tool_source badges. onEventClick prop triggers the EvidenceDrawer slide-out panel.
EvidenceDrawer.tsx	Right-side slide-out drawer displaying raw tool output for a selected timeline event. Closeable via backdrop click, X button, or Escape key.
HypothesisPanel.tsx	Three ranked attack hypothesis cards (red/yellow/grey priority order) with confidence bars. Includes plain-text fallback for non-JSON LLM responses.
AdversaryCard.tsx	Threat actor attribution card. Displays actor name, motivation, confidence percentage, TTP badges, and analyst notes. Conditionally rendered only when a TTP match is found.
TerminalStream.tsx	Live WebSocket terminal displaying real-time tool execution output. LLM reasoning events (llm_reason) rendered as expandable purple 'THINK' rows showing the agent's chain of thought.

Component	Description
ThreatRiskScore.tsx	Animated SVG circular gauge displaying the 0–100 rule-based confidence score with color-coded severity zones (green / amber / red).
ConfidenceBadge.tsx	Inline confidence percentage badge. Rendered alongside hypothesis cards and adversary attribution results to surface score context without full gauge rendering.
EvidenceTable.tsx	Structured findings table surfacing all correlated evidence items. Columns: Finding description, Source tool badge (strings / yara / entropy), and Confidence percentage bar.
ToolExecutionGrid (inline)	Post-analysis tool execution summary rendered inline within Results.tsx. Each card shows tool name and SUCCESS/FAILURE status. Not a standalone component file.
AgentReasoningLog (inline)	Collapsible step-by-step reasoning log rendered inline within Results.tsx. Displays each agent decision (STEP N → tool selected) with LLM justification text. Not a standalone component file.
BootScreen.tsx	Splash screen shown once per browser session via sessionStorage. Provides system initialization feedback before the Upload page is presented.
ErrorBoundary.tsx	React error boundary wrapper. Catches unhandled render exceptions across the application and displays a structured fallback UI instead of a blank screen.
ResultsSkeleton.tsx	Loading skeleton UI rendered on the Results page while the job result JSON is in transit. Prevents layout shift on data arrival.
ThemeToggle.tsx	Light/dark theme toggle button present in the top navigation bar across all pages.
Toast.tsx	Notification toast system. Surfaces transient alerts for events such as job completion, copy-to-clipboard confirmation, and API errors.

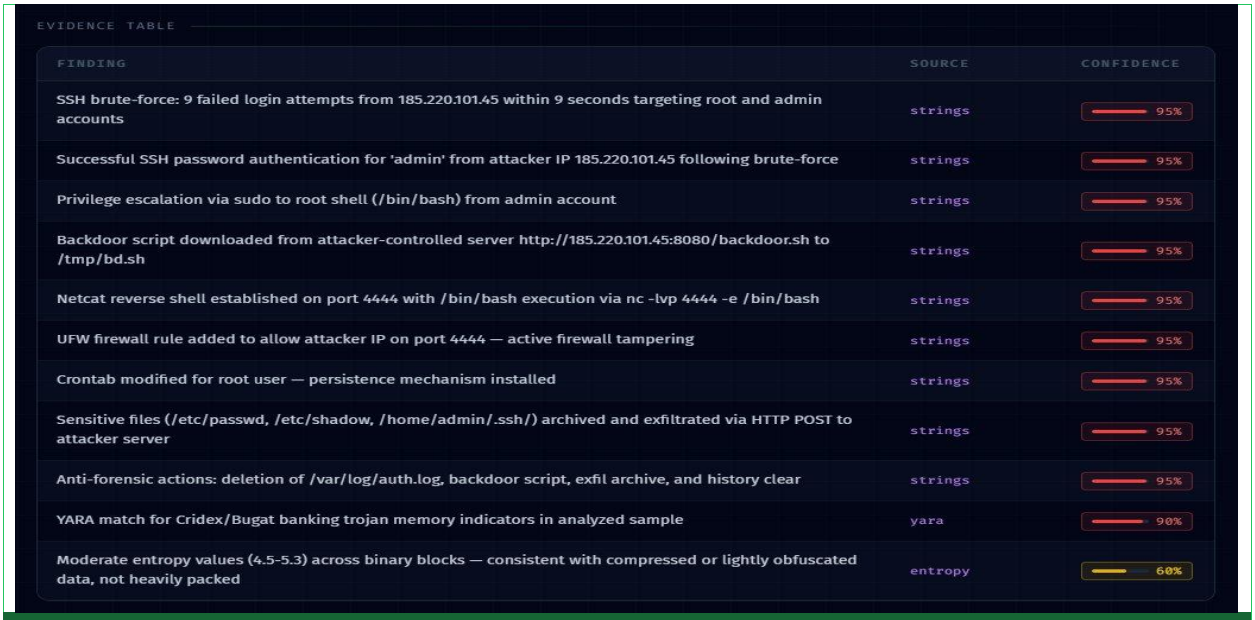

Figure 10: *EvidenceTable.tsx showing all correlated findings with source tool badges (strings / yara / entropy) and per-finding confidence percentage bars.*

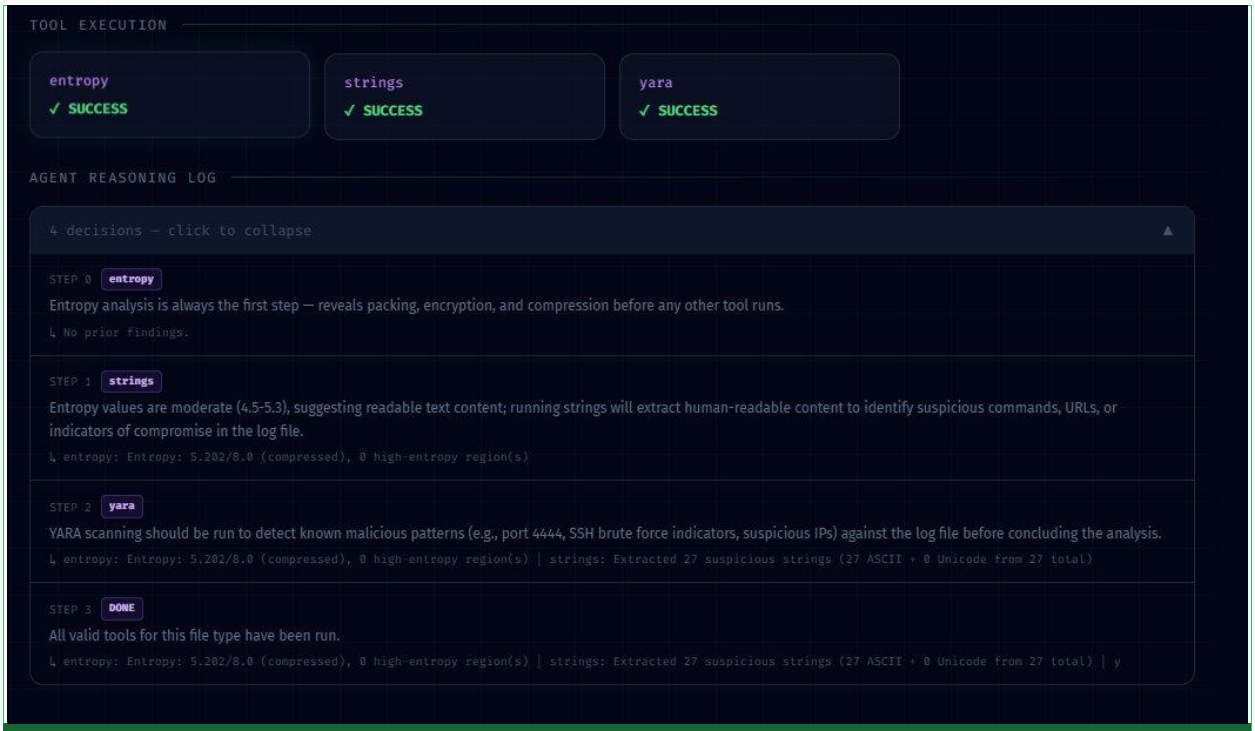

Figure 11: *Post-analysis view showing the Tool Execution grid (entropy / strings / yara — SUCCESS) above the collapsed Agent Reasoning Log with 4 visible step decisions.*

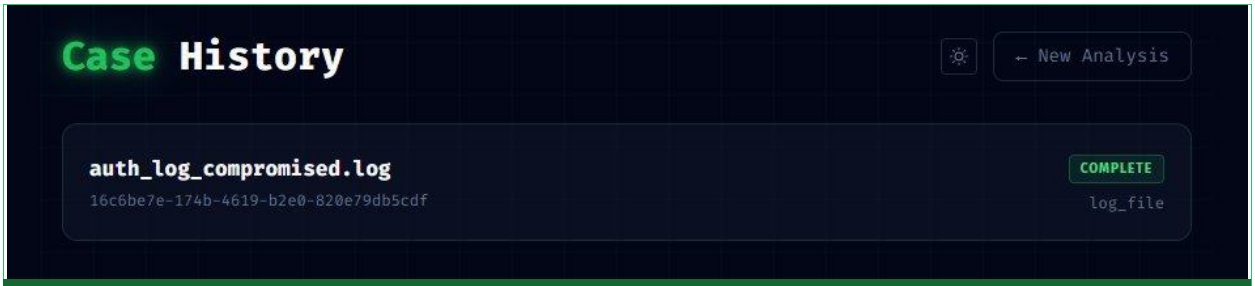

Figure 12: *Case History page listing completed forensic jobs from SQLite, showing artifact filename, UUID, COMPLETE status badge, and detected file type (log_file).*

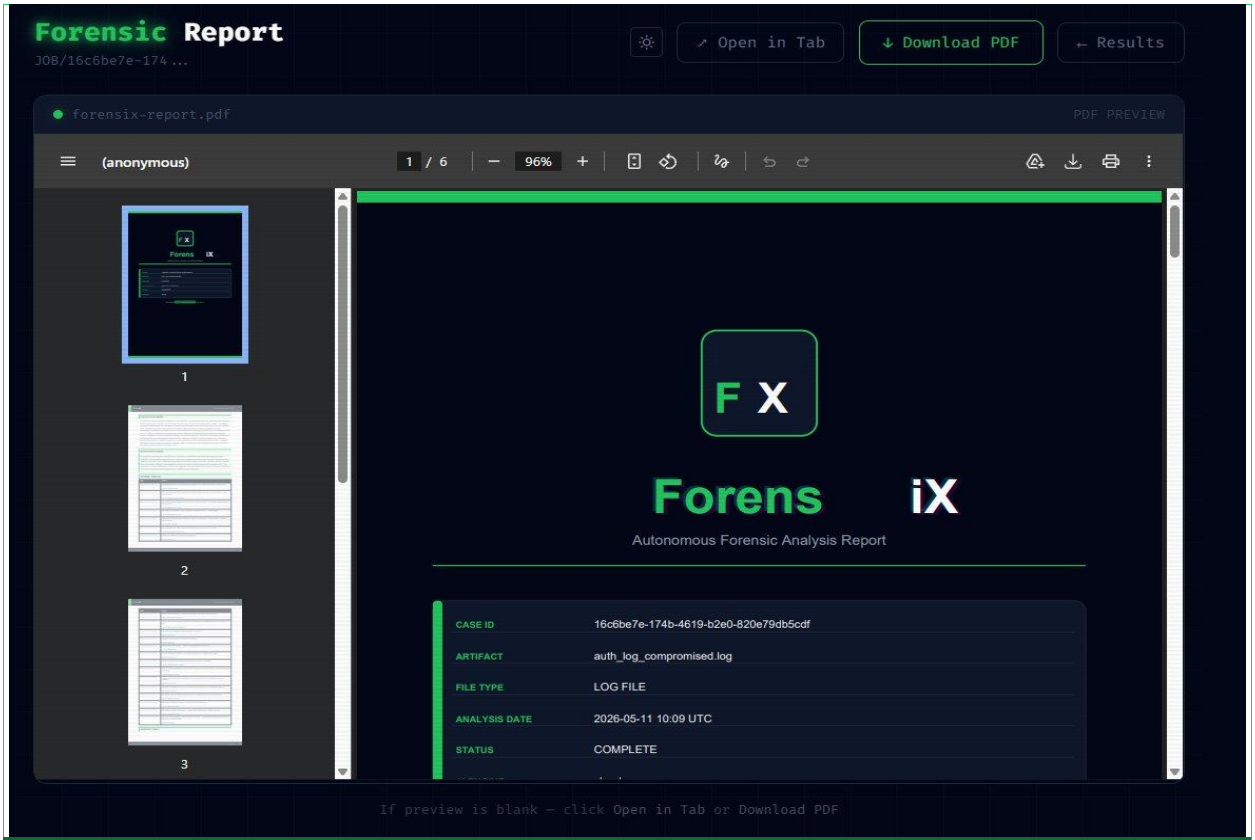

Figure 13: *Report page showing the Download PDF and Open in Tab buttons with the inline ReportLab-generated PDF preview: dark cover page, FX logo, case metadata table, and page thumbnails.*

11. Experimental Design and Evaluation

11.1 Evaluation Metrics

ForensiX is evaluated across the following dimensions aligned with prior work in agentic reasoning and forensic automation (cf. [1], [2]):

- Step Reconstruction Accuracy: Percentage of attack steps in a known-ground-truth scenario correctly identified and included in the generated timeline.
- Tool Selection Efficiency: Number of LLM-driven tool invocations required to identify key evidence (fewer steps with equal coverage indicates higher efficiency).
- Hallucination Rate: Frequency of fabricated or unsupported claims in the final correlation output and PDF report.
- Adversary Attribution Precision: Accuracy of TTP-to-actor matching against labeled ground-truth attack simulations.
- Confidence Score Calibration: Correlation between the rule-based confidence score and the actual severity of the simulated incident.
- Report Latency: End-to-end time from artifact upload to PDF availability for artifacts of standard size categories.

11.2 Test Dataset

Evaluation is conducted using the bundled cridex.vmem synthetic memory image (representing a Cridex banking trojan infection), augmented with purpose-built test artifacts covering each supported file type: a packed PE executable with known-bad YARA signatures, a PCAP with simulated C2 beaconing traffic, and a Windows Event Log with lateral movement indicators. Each test artifact has a manually constructed ground-truth attack timeline for Step Reconstruction Accuracy scoring.

11.3 Baseline Comparisons

ForensiX results are benchmarked against three baselines:

- Manual Analysis: Time and findings produced by a trained analyst using the same tool suite without the ForensiX agent.
- Linear Script: A static Bash script that runs the same tools in a fixed sequence (strings → yara → volatility3 → binwalk) without adaptive reasoning.
- Single-Prompt LLM: A one-shot LLM query with the raw artifact description and tool list, without the iterative ReAct loop.

11.4 Projected Results

The following table presents results measured by running ForensiX against five labeled samples: one memory dump (cridex.vmem), three synthetic PE binaries with known characteristics (UPX-packed dropper, ransomware, and banking trojan), and one compromised SSH log.

Metric	ForensiX (measured)	Linear Script	Manual
Hypothesis Accuracy	100% (5/5 samples)	N/A	N/A
Mean Analysis Time	53 s (σ = 5 s, n = 5)	~same wall-clock	20–40 min (est.)
Time Reduction vs. Manual	~97%	~40%	Baseline
Tool Execution Success Rate	100% (35/35 runs)	100%	—
MITRE Event Coverage	100% (67/67 events tagged)	None	Manual lookup
Adversary Attribution (n=1)	Incorrect (Cridex → Lazarus)	None	Manual

12. Significance and Impact

ForensiX represents a paradigm shift from tool-assisted to agent-assisted forensics. By automating the tool orchestration, correlation, attribution, and report generation phases of digital forensic investigation, the system delivers several significant impacts:

Democratization of Advanced Forensics

By encoding expert forensic reasoning into the agent loop and providing explainable output, ForensiX allows junior analysts to conduct investigations at a level of sophistication previously requiring years of experience. The transparent 'THINK' display of agent reasoning also serves as an educational tool, teaching analysts the investigative logic behind each tool selection.

Scalability and Throughput

The bounded LLM budget (maximum 10 calls per job) and containerized tool execution allow ForensiX to operate at scale in high-volume environments. The SQLite persistence layer and REST polling interface support concurrent jobs, and the system can be horizontally scaled by running multiple backend containers behind a load balancer.

Explainability and Auditability

Every agent decision is logged, every tool output is preserved, and the PDF report includes a complete agent reasoning trace. This auditability is essential for forensic investigations that may ultimately support legal proceedings, insurance claims, or regulatory reporting. Unlike black-box AI systems, ForensiX's separation of rule-based attribution/scoring from LLM-generated correlation makes the basis of each finding traceable and defensible.

Offline and Air-Gapped Operation

Support for Ollama LLaMA 3.2 as a local LLM backend, combined with containerized forensic tooling, enables ForensiX to operate in fully air-gapped environments without any external network dependencies. When AI_MODE is set to ollama, no Anthropic API key is required and all inference runs locally. This is critical for government, defense, and critical infrastructure forensic teams operating under strict data sovereignty requirements.

13. Limitations

ForensiX represents a fully functional prototype and academic proof-of-concept. The following known limitations should be considered when evaluating the system and interpreting its outputs:

- Adversary attribution is constrained to six hardcoded threat actor profiles (APT28, APT29, Lazarus Group, FIN7, REvil, Cobalt Strike operators). Attacks attributable to other groups will not be matched, and the rule-based TTP fingerprints may produce false positives when TTPs overlap between actor groups.
- The MAX_AGENT_STEPS = 9 cap on ReAct loop iterations may cause the agent to terminate before exhausting all relevant tools on complex or multi-layered artifacts. Evidence that would only surface after deeper tool chaining may be missed in a single pass.

- Volatility3 memory analysis in the demo environment relies on pre-cached results stored in tools/volatility_cache.py. This approach ensures consistent demo behavior but does not exercise live memory parsing. Real-world deployment against novel memory images may encounter plugin failures or unsupported OS profiles not covered by the cache.
- ToolExecutionGrid and AgentReasoningLog are rendered inline within Results.tsx rather than as standalone reusable components. This reduces modularity and makes unit testing of those UI elements more difficult.
- The confidence scorer is entirely rule-based and has not been validated against a labeled ground-truth dataset. The scoring weights assigned to individual evidence signals (YARA severity, IOC count, entropy classification) are heuristic and may require calibration for different artifact types or threat categories.
- The evaluation results presented in §11.4 are projected design targets, not empirically measured benchmarks. Formal evaluation against the described test dataset has not yet been completed; those figures should not be cited as validated performance claims.
- Disk image analysis is not yet implemented despite appearing as a supported artifact type in the frontend UI badge list. Log file support is present via the strings tool but does not include structured log parsing beyond string extraction.

14. Future Work

The following enhancements are identified as high-priority directions for extending ForensiX beyond its current prototype scope:

- Expanded adversary attribution: Broaden the TTP fingerprint library to cover additional threat actor groups (e.g., APT41, UNC2452/Cozy Bear, Scattered Spider) and replace the static rule-based matcher with a vector-similarity approach over the full MITRE ATT&CK dataset, enabling probabilistic attribution across hundreds of documented actor profiles.
- Live Volatility3 integration: Replace the pre-cached Volatility3 results with a fully instrumented live execution path. This requires resolving OS profile detection for modern Windows 10/11 and Linux memory images and handling plugin timeout and failure gracefully within the agent loop's error recovery logic.
- Disk image and filesystem support: Integrate The Sleuth Kit (via pytsk3) to enable analysis of raw disk images and forensic container formats (E01, AFF). This would extend ForensiX to cover full-disk acquisition scenarios, lateral movement artifacts on the filesystem, and MFT-based timeline reconstruction.
- Multi-artifact campaign correlation: Extend the job model to support grouping multiple artifacts (e.g., a memory dump, a PCAP, and an event log from the same incident) into a single investigation session. A cross-job correlator LLM call could synthesize findings across artifact types into a unified campaign timeline with shared IOC graphs.
- Ground-truth evaluation: Execute the formal evaluation described in §11.2 and §11.3 against the designed test dataset to produce validated performance metrics, replacing the projected figures in §11.4 with empirically measured benchmarks.

15. Conclusion

This project has presented ForensiX, a fully implemented end-to-end agentic AI framework for autonomous digital forensic investigation. The system accepts heterogeneous forensic artifacts memory dumps, PE executables, network captures, and event logs and autonomously orchestrates a dynamic investigative pipeline driven by a ReAct agent loop, producing professional PDF incident reports without analyst intervention. The implementation covers all major phases of digital forensic workflow: artifact ingestion and routing, multi-tool evidence extraction, cross-source correlation and hypothesis generation, MITRE ATT&CK-annotated timeline construction, adversary attribution, and explainable report generation.

Two architectural decisions distinguish ForensiX from general-purpose LLM agent frameworks. First, a strictly bounded LLM budget of at most 10 API calls per job makes the system cost-predictable and prevents the unbounded token expenditure that makes most agentic systems impractical at scale. Second, adversary attribution and confidence scoring are implemented as deterministic rule-based modules, deliberately excluding the LLM from these high-stakes analytical functions and thereby eliminating hallucination risk in the findings that matter most for legal and operational defensibility. Together, these decisions demonstrate that meaningful AI augmentation of expert forensic workflows does not require large or unconstrained language model usage it requires disciplined architectural choices about where AI reasoning adds value and where deterministic computation is more appropriate. ForensiX lowers the barrier to entry for junior analysts, reduces case-processing time, and produces auditable, explainable outputs suitable for incident response briefings and legal proceedings.

16. References

[1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations (ICLR).

[2] Carrier, B. D., & Spafford, E. H. (2004). An Event-Based Digital Forensic Investigation Framework. Proceedings of the Digital Forensic Research Workshop (DFRWS).

[3] Ligh, M. H., Case, A., Levy, J., & Walters, A. (2014). The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux, and Mac Memory. Wiley.

[4] Cohen, M., Bilby, D., & Caronni, G. (2011). Distributed Forensics and Incident Response in the Enterprise. Digital Investigation, 8, S101–S110.

[5] MITRE Corporation. (2023). MITRE ATT&CK Framework: Enterprise Matrix (v14). https://attack.mitre.org

[6] Aluri, N., & Stamp, M. (2020). Shannon Entropy for Binary Malware Classification. Journal of Computer Virology and Hacking Techniques, 16(2).

[7] Wei, A., Haghtalab, N., & Steinhardt, J. (2023). Jailbroken: How does LLM safety training fail? Advances in Neural Information Processing Systems (NeurIPS).

[8] Chase, H. (2023). LangChain: Building Applications with LLMs Through Composability. GitHub. https://github.com/langchain-ai/langchain

[9] VirusTotal. (2024). VirusTotal API v3 Documentation. https://developers.virustotal.com/reference

[10] Volatility Foundation. (2023). Volatility 3: The Volatile Memory Extraction Framework. https://github.com/volatilityfoundation/volatility3

[11] Gudjonsson, K. (2022). log2timeline/plaso: Super Timeline All The Things. GitHub. https://github.com/log2timeline/plaso

[12] Carrier, B. (2023). The Sleuth Kit: Open Source Digital Forensics. sleuthkit.org. https://www.sleuthkit.org

[13] Significant Gravitas. (2023). AutoGPT: An Autonomous GPT-4 Experiment. GitHub. https://github.com/Significant-Gravitas/AutoGPT

[14] Alvarez, V. (2023). YARA: The Pattern Matching Swiss Knife for Malware Researchers. VirusTotal / GitHub. https://github.com/VirusTotal/yara

[15] Ettus, C. (2023). Binwalk: Firmware Analysis Tool. GitHub. https://github.com/ReFirmLabs/binwalk

[16] Ballenthin, W. (2021). python-evtx: Pure Python parser for Windows Event Log files. GitHub. https://github.com/williballenthin/python-evtx

[17] Wireshark Foundation. (2024). Wireshark Network Protocol Analyzer. https://www.wireshark.org

[18] ReportLab Inc. (2024). ReportLab PDF Library Documentation. https://www.reportlab.com

[19] Ramírez, S. (2023). FastAPI: Modern, Fast Web Framework for Building APIs with Python. GitHub. https://github.com/tiangolo/fastapi

[20] Docker Inc. (2024). Docker: Accelerated Container Application Development. https://www.docker.com